

3 Process

Process Concept

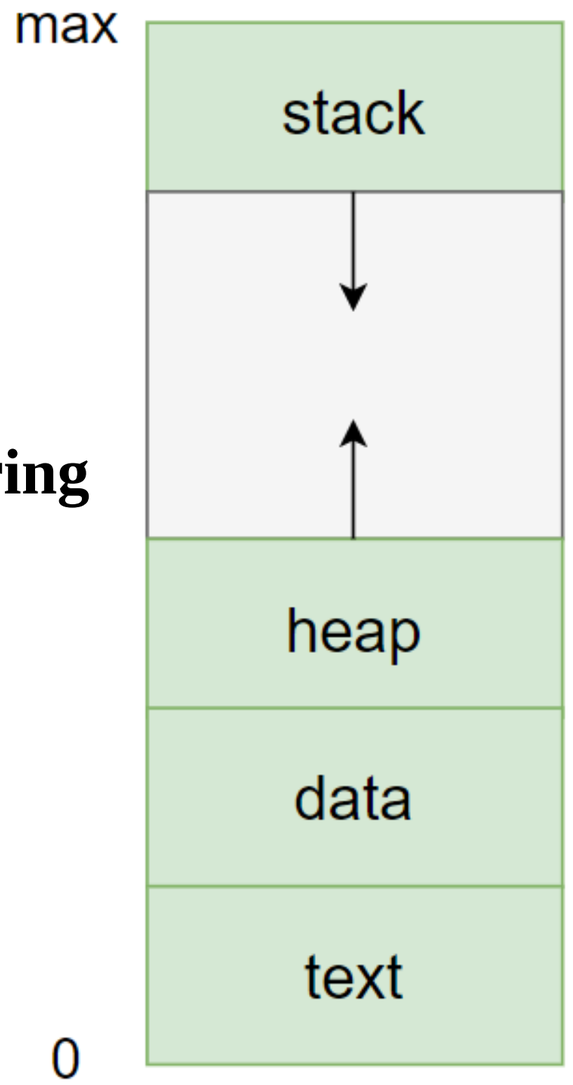
- **An operating system executes a variety of programs:**
  - Batch system – **jobs**
  - Time-shared systems – **user programs** or **tasks**
- **The terms job and process are used sometimes interchangeably**
- **Process – a program in execution; process execution must progress in sequential fashion**

Process Concept

- **Program is *passive* entity stored on disk (executable file), process is *active***
  - Program becomes process when executable file loaded into memory
- **Execution of program started via GUI mouse clicks, command line entry of its name, etc**
- **One program can be several processes**
  - Consider multiple users executing the same program

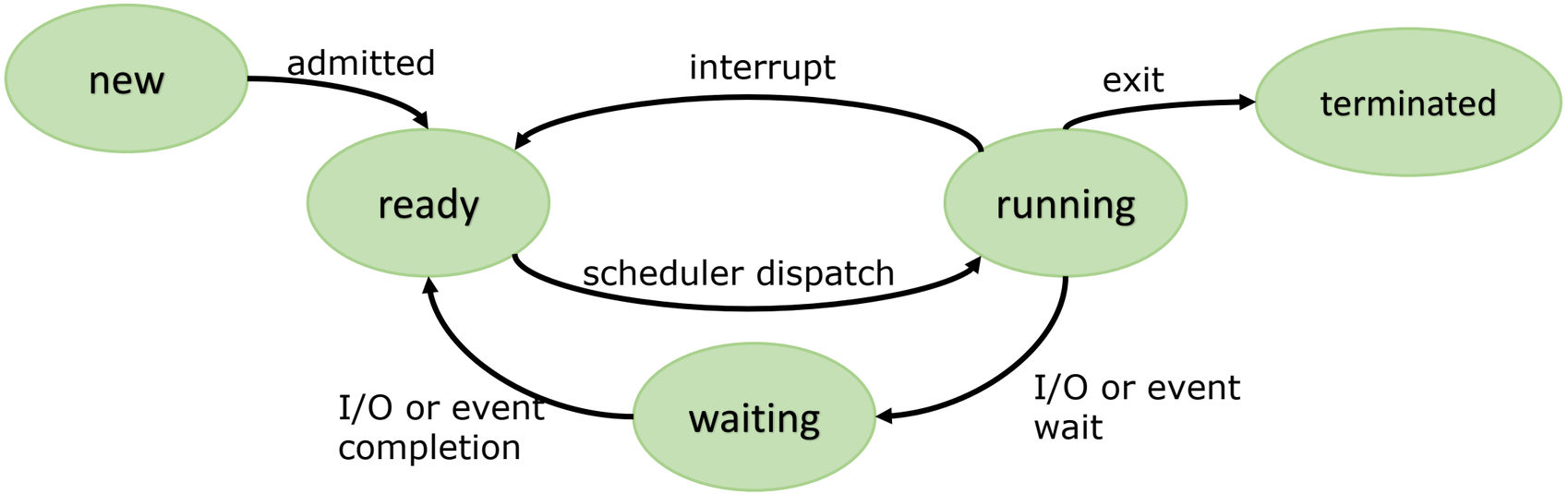
Process in Memory

- **The program code, also called text section**
- **Data section containing global variables**
- **Stack containing temporary data**
  - Function parameters, return addresses, local variables
- **Heap containing memory dynamically allocated during run time**



Process State

- **As a process executes, it changes state**
  - **new**: The process is being created
  - **ready**: The process is waiting to be assigned to a processor
  - **running**: Instructions are being executed
  - **waiting**: The process is waiting for some event to occur
  - **terminated**: The process has finished execution

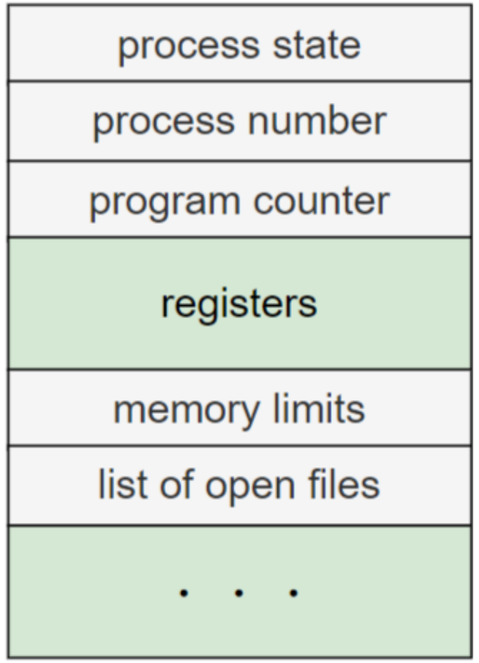


Process Control Block (PCB)

Information associated with each process

(also called task control block)

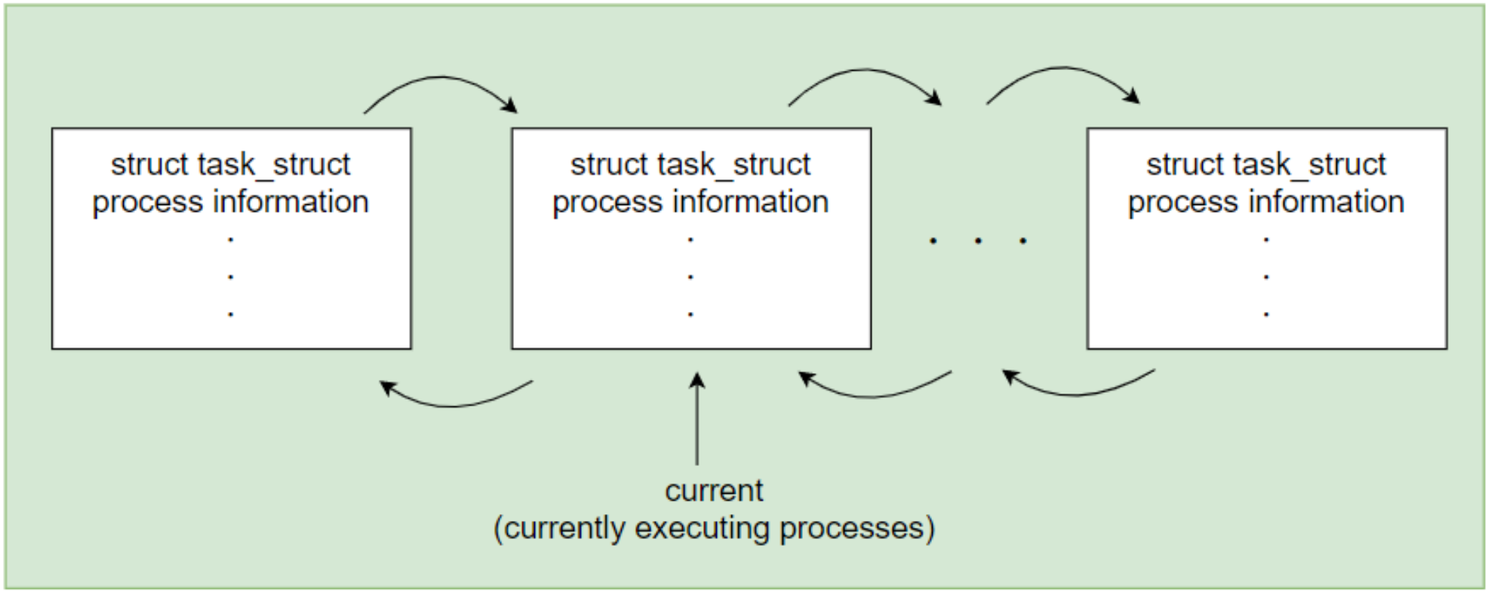
- **Process state – running, waiting, etc**
- **Program counter – location of instruction to next execute**
- **CPU registers – contents of all process-centric registers**
- **CPU scheduling information- priorities, scheduling queue pointers**
- **Memory-management information – memory allocated to the process**
- **I/O status information – I/O devices allocated to process, list of open files**
- **Accounting information – CPU used, clock time elapsed since start, time limits**



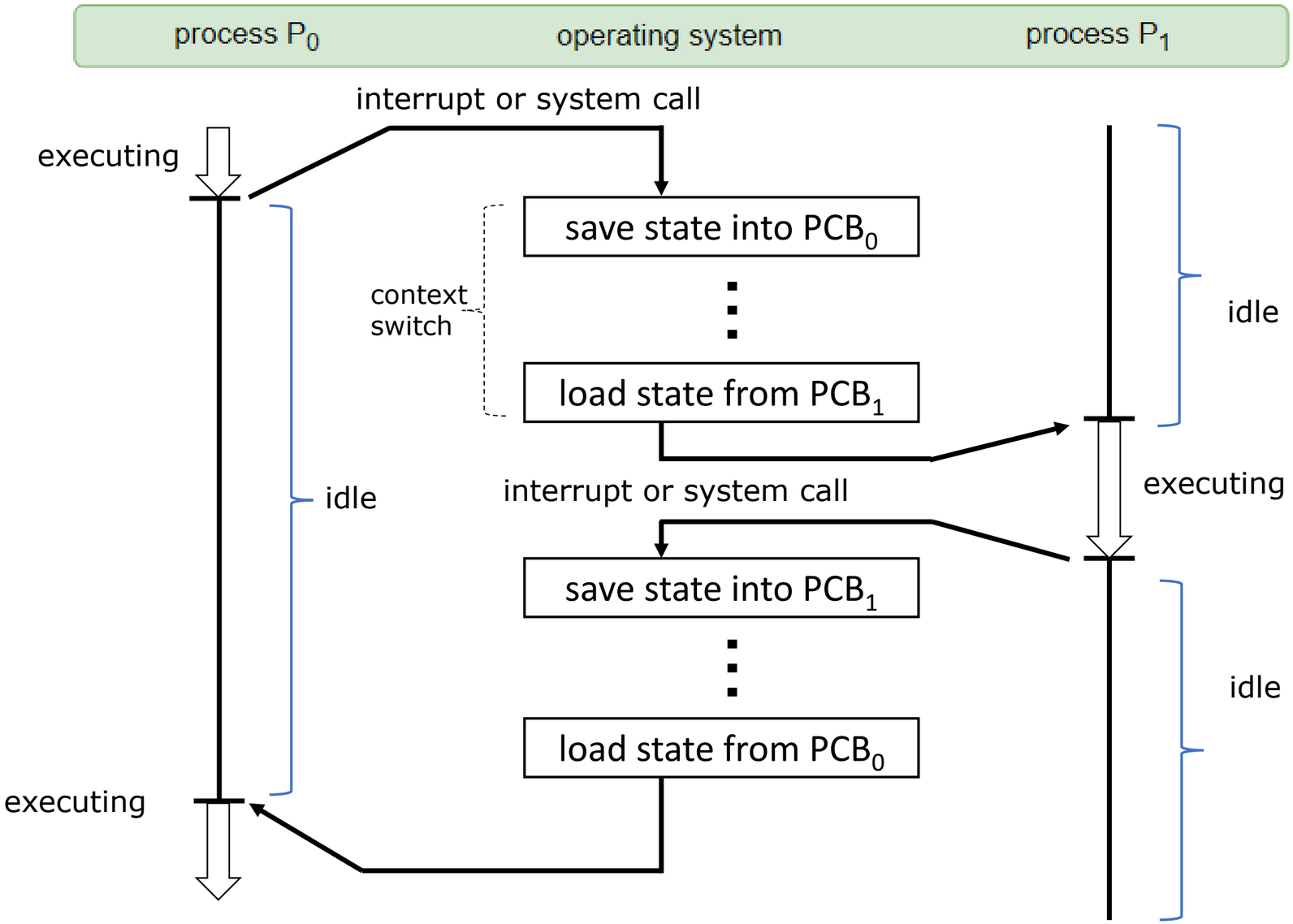
Process Representation in Linux

- **Represented by the C structure `task_struct`**

```
1  pid_t pid; /* process identifier */
2  long state; /* state of the process */
3  unsigned int time_slice /* scheduling information */
4  struct task_struct *parent; /* this process's parent */
5  struct list_head children; /* this process's children */
6  struct files_struct *files; /* list of open files */
7  struct mm_struct *mm; /* address space of this process */
```



CPU Switch From Process to Process



Context Switch

- **When CPU switches to another process, the system must save the state of the old process and load the saved state for the new process via a context switch**

- **Context of a process represented in the PCB**

- **Context-switch time is overhead; the system does no useful work while switching**
  - The more complex the OS and the PCB ➔ longer the context switch

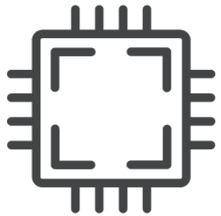
- **Time dependent on hardware support**
  - Some hardware provides multiple sets of registers per CPU ➔ multiple contexts loaded at once

Process Scheduling

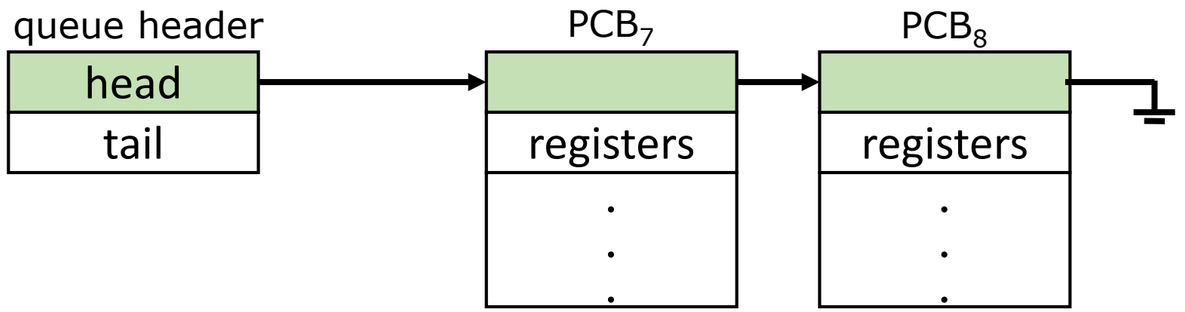
- **Maximize CPU use, quickly switch processes onto CPU for time sharing**
- **Process scheduler selects among available processes for next execution on CPU**
- **Maintains scheduling queues of processes**
  - **Job queue** – set of all processes in the system
  - **Ready queue** – set of all processes residing in main memory, ready and waiting to execute
  - **Device queues** – set of processes waiting for an I/O device
  - Processes migrate among the various queues

Ready Queue And Various I/O Device Queues

CPU



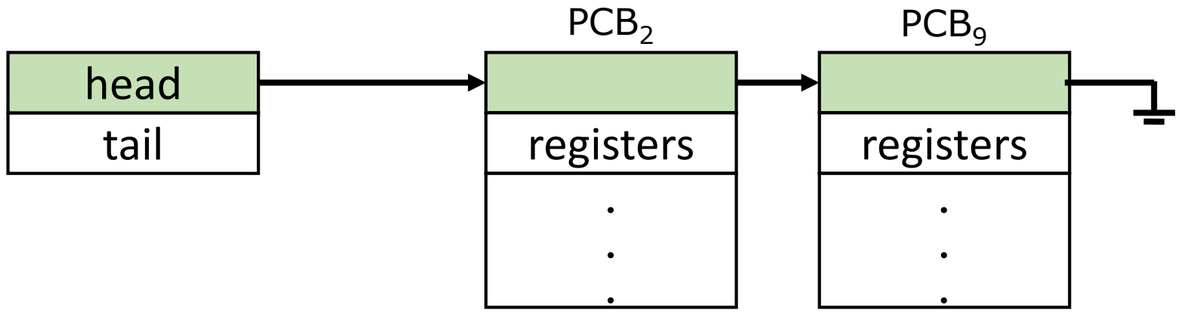
ready queue



disk



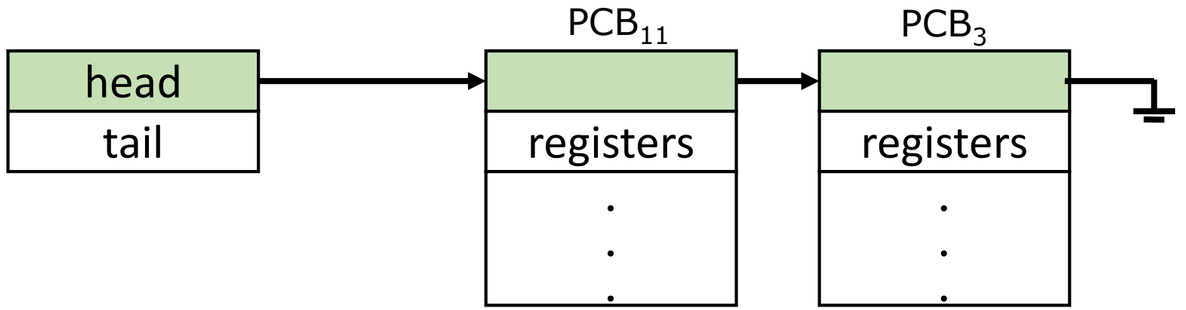
disk unit 0



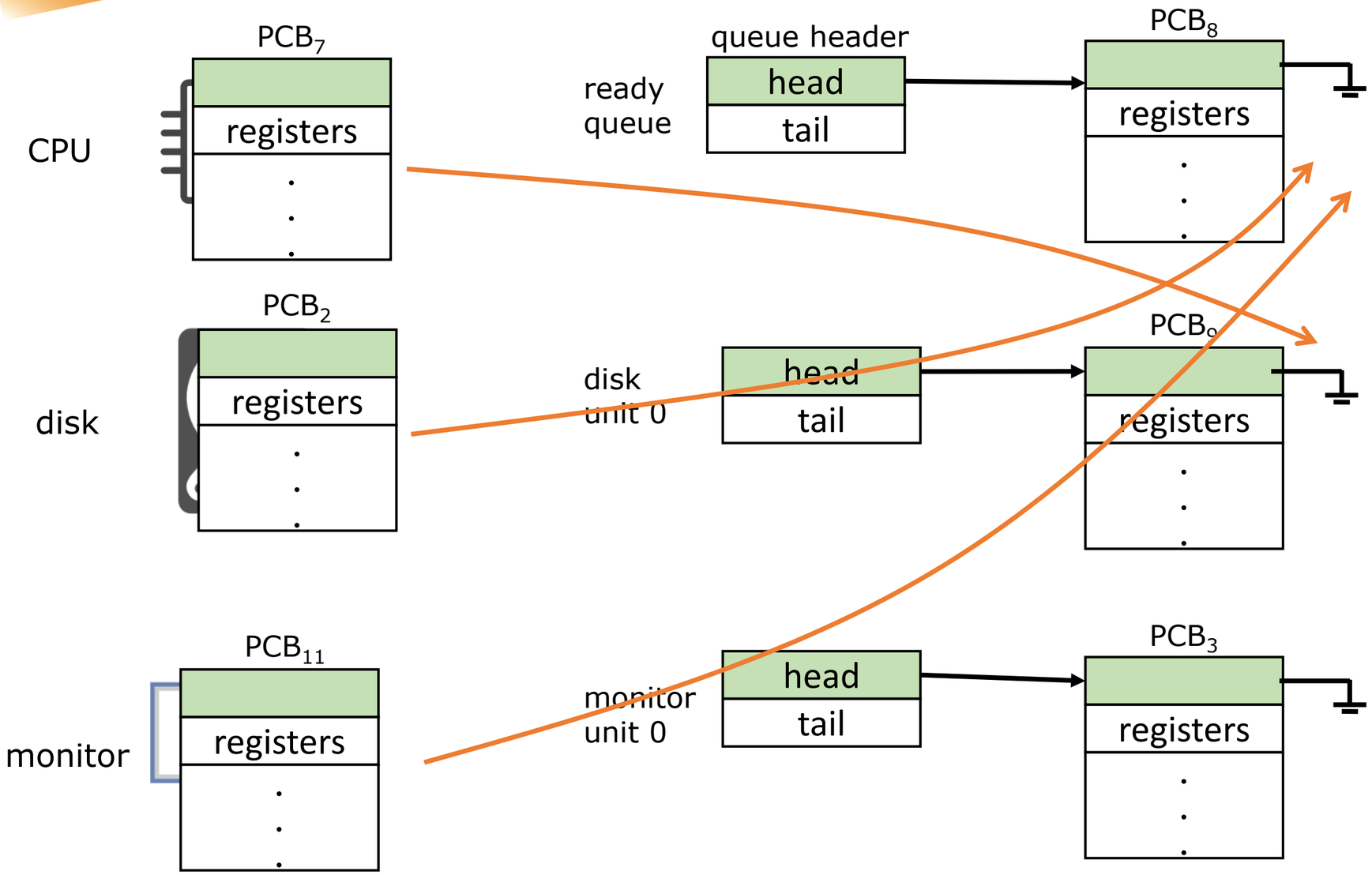
monitor



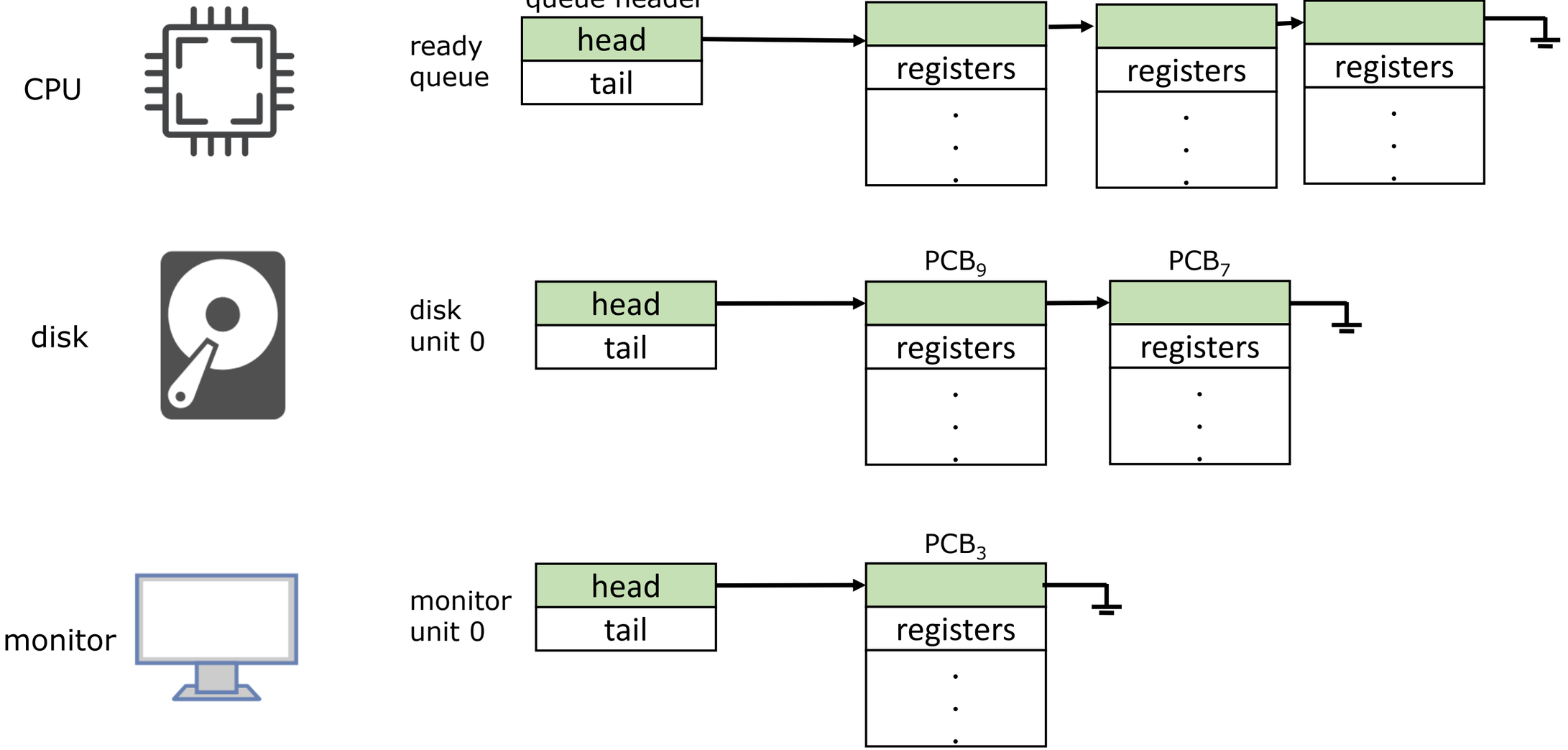
monitor unit 0



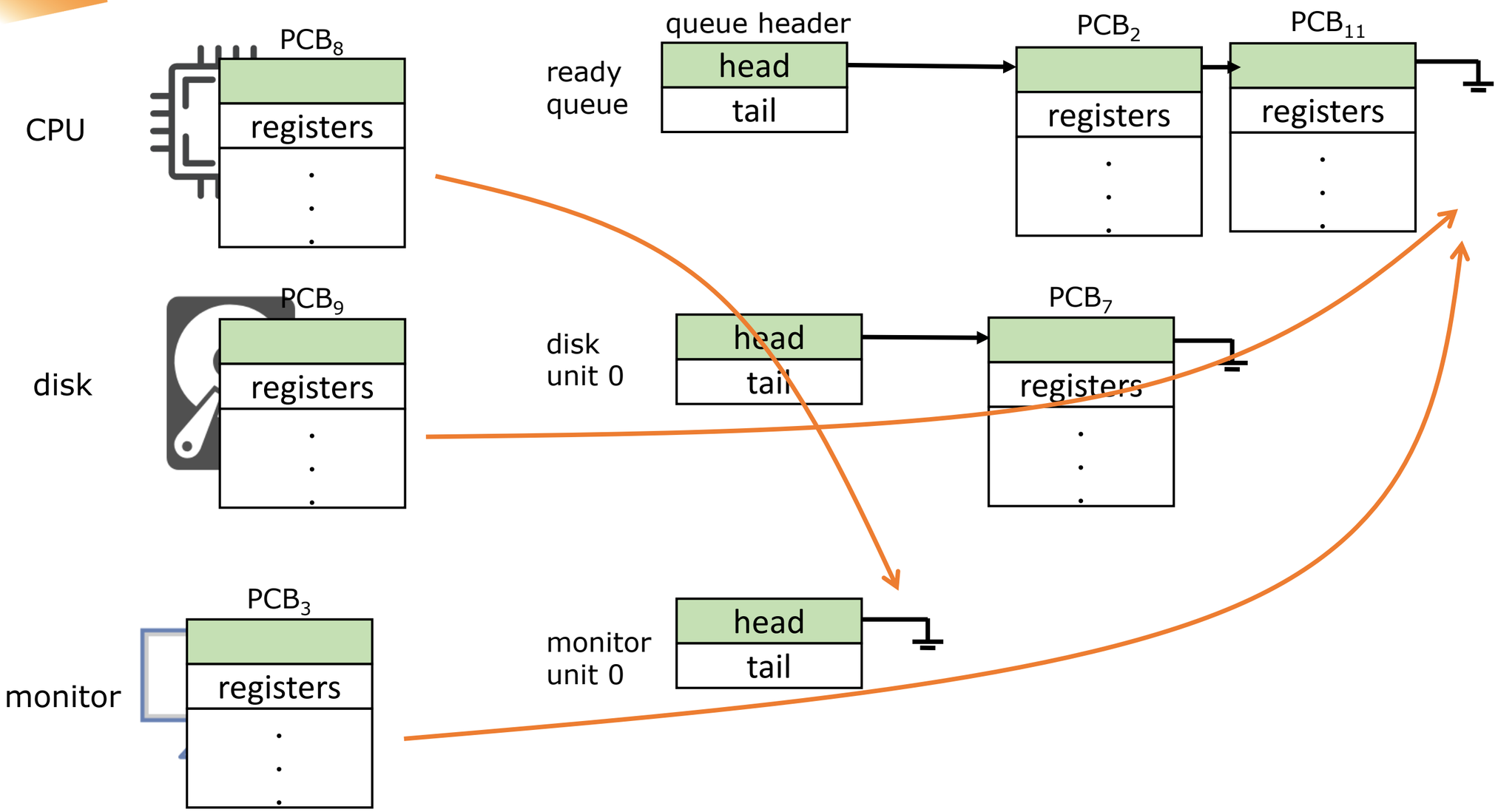
Ready Queue And Various I/O Device Queues



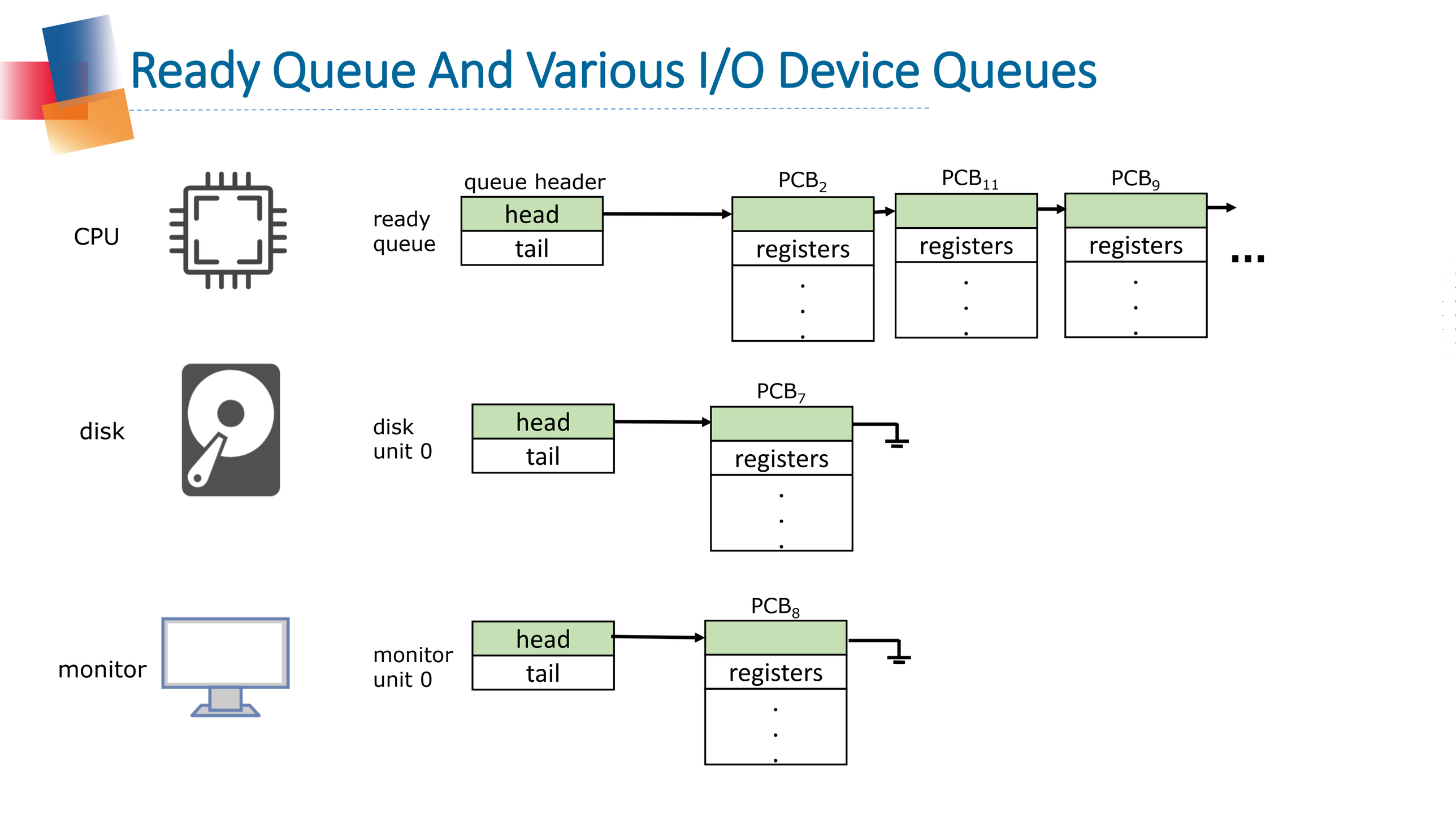
Ready Queue And Various I/O Device Queues



Ready Queue And Various I/O Device Queues

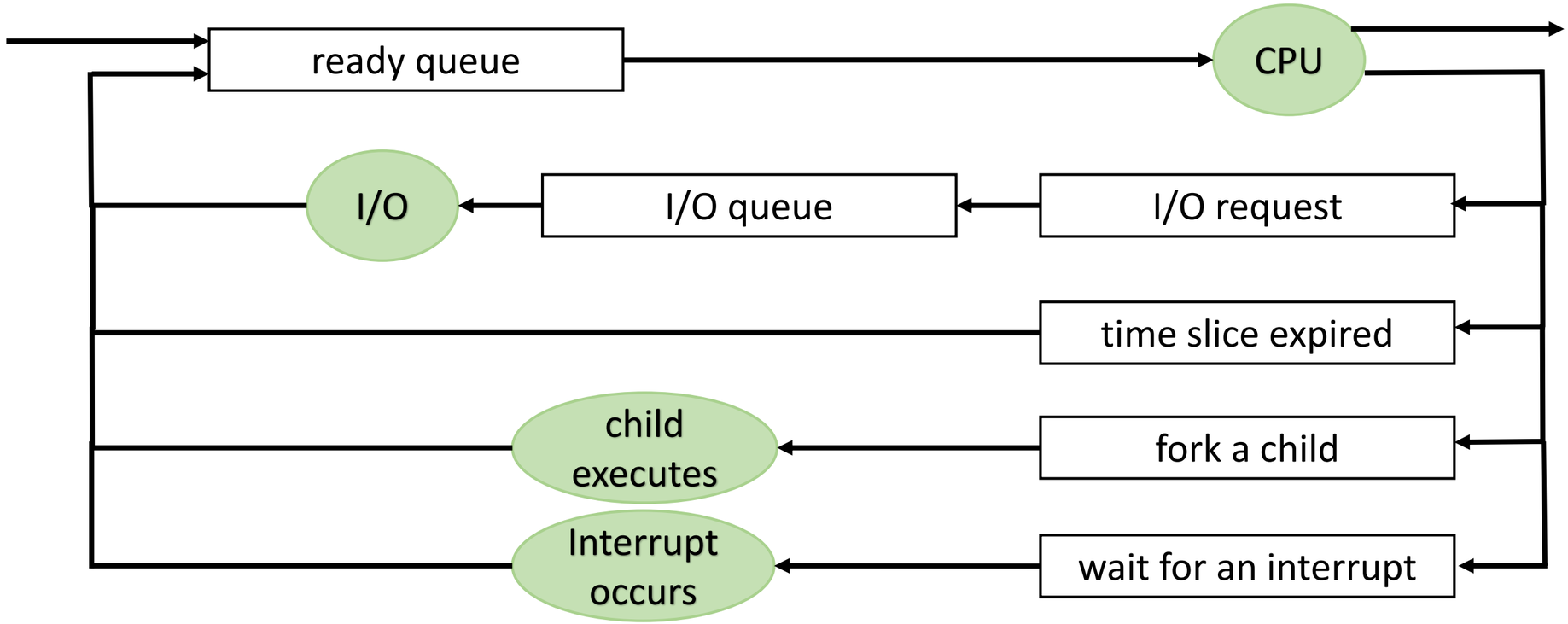


Ready Queue And Various I/O Device Queues



Representation of Process Scheduling

Queuing diagram represents queues, resources, flows

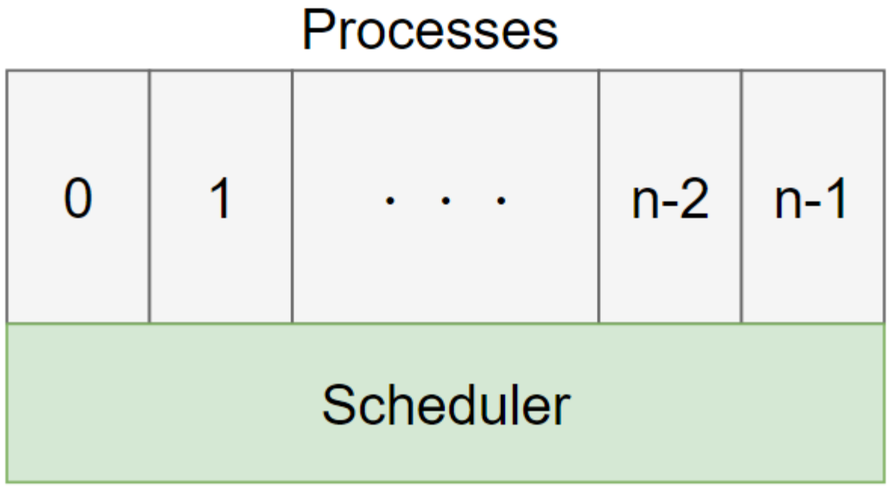


Schedulers

- **Long-term scheduler (or job scheduler) – selects which processes should be brought into the ready queue**
- **Short-term scheduler (or CPU scheduler) – selects which process should be executed next and allocates CPU**
  - Sometimes the only scheduler in a system
- **Short-term scheduler is invoked very frequently (milliseconds) ⇒ (must be fast)**
- **Long-term scheduler is invoked very infrequently (seconds, minutes) ⇒ (may be slow)**
- **The long-term scheduler controls the degree of multiprogramming**
- **Processes can be described as either:**
  - **I/O-bound process** – spends more time doing I/O than computations, many short CPU bursts
  - **CPU-bound process** – spends more time doing computations; few very long CPU bursts
- **Long-term scheduler strives for good *process mix***

Process and Schedulers

- **Lowest layer of process-structured OS**
  - handles interrupts, scheduling
- **Above that layer are sequential processes**



Multitasking in Mobile Systems

- **Some systems / early systems allow only one process to run, others suspended**
- **Due to screen real estate, user interface limits iOS provides for**
  - Single **foreground** process- controlled via user interface
  - Multiple **background** processes– in memory, running, but not on the display, and with limits
  - Limits include single, short task, receiving notification of events, specific long-running tasks like audio playback
- **Android runs foreground and background, with fewer limits**
  - Background process uses a **service** to perform tasks
  - Service can keep running even if background process is suspended
  - Service has no user interface, small memory use

Operations on Processes

- **System must provide mechanisms for process creation, termination, and so on.**

Starting A Process

- **Command interpreter runs user programs**
- **Tells OS to start another process**
- **Traditionally means a FORK OS call**
- **Calling process is *Parent***
- **OS copies the running process**
- **New process is *Child***
- **EXEC OS call loads new program**

Process Creation

- **Parent process create children processes, which, in turn create other processes, forming a tree of processes**
- **Generally, process identified and managed via a process identifier (pid)**
- **Resource sharing options**
  - Parent and children share all resources
  - Children share subset of parent's resources
  - Parent and child share no resources
- **Execution options**
  - Parent and children execute concurrently
  - Parent waits until children terminate

Process Creation (Cont.)

Address space

- Child duplicate of parent
- Child has a program loaded into it

UNIX examples

- **`fork()`** system call creates new process
- **`exec()`** system call used after a **`fork()`** to replace the process' memory space with a new program

C Program Forking Separate Process

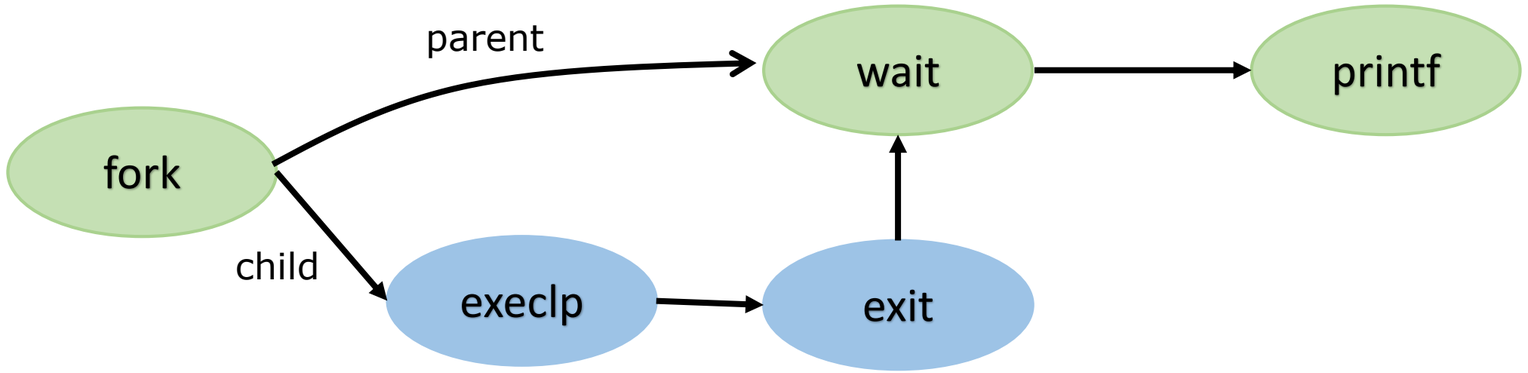
```
#include <sys/types.h>
#include <stdio.h>
#include <unistd.h>

int main()
{
pid_t pid;

    /* fork a child process */
    pid = fork();

    if (pid < 0) { /* error occurred */
       fprintf(stderr, "Fork Failed");
       return 1;
    }
    else if (pid == 0) { /* child process */
       execlp("/bin/ls","ls",NULL);
    }
    else { /* parent process */
       /* parent will wait for the child to complete */
       wait(NULL);
       printf("Child Complete");
    }

    return 0;
}
```



fork()

- **The `fork()` system call will spawn a new child process which is an identical process to the parent except that has a new system process ID.**
  - The process is copied in memory from the parent and a new process structure is assigned by the kernel.
  - The return value of the function is which discriminates the two threads of execution.
  - A zero is returned by the fork function in the child's process.
- **The environment, resource limits, umask, controlling terminal, current working directory, root directory, signal masks and other process resources are also duplicated from the parent in the forked child process.**

```c
main() {
        int return_value;
        printf("Forking process \n");
        return_value = fork();
        printf("The process id is %d and return value is %d \n",
                getpid(), return_value);
        execl("/bin/ls/", "ls", "-l", 0);
        printf("This line is not printed \n");
    }
```

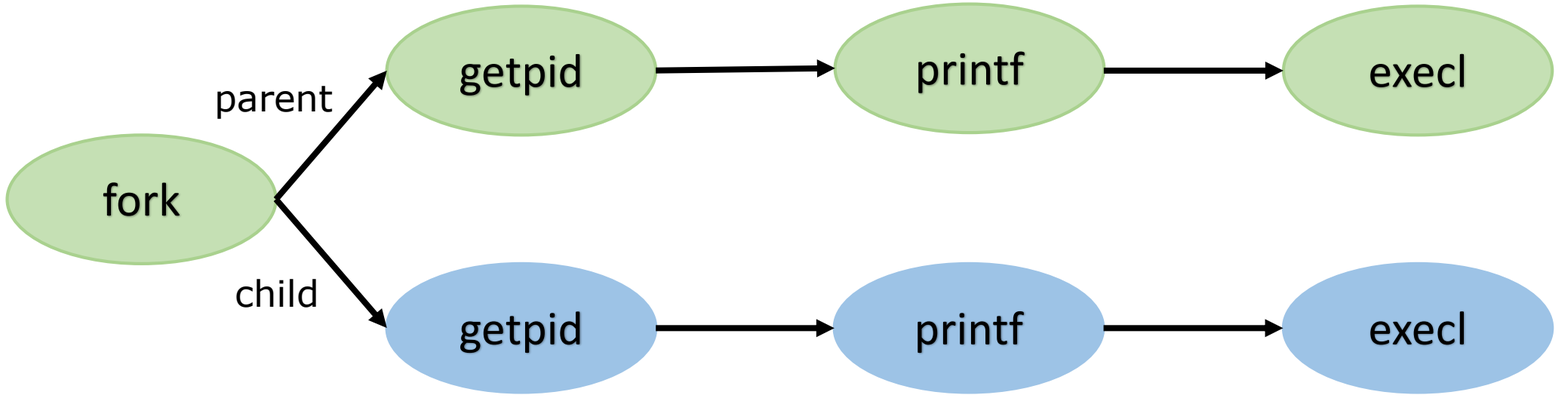
The Output of this would be:

Forking process

The process id is 6753 and return value is 6754

The process id is 6754 and return value is 0

two lists of files in current directory



wait()

- **The parent process will often want to wait until all child processes have been completed. this can be implemented with the `wait()` function call.**
  - **`wait()`** return the pid of the child or -1 for an error.
  - The exit status of the child is returned to **`status_location`**.

```
int wait (int *status_location)
```

- **`wait()`: Blocks calling process until the child process terminates.**
  - If child process has already terminated, the **`wait()`** call returns immediately.
  - If the calling process has multiple child processes, the function returns when one returns.

execl(),execlp()- exec()family

- **The function call "`execl()`" initiates a new program in the same environment in which it is operating. An executable (with fully qualified path. i.e. `/bin/ls`) and arguments are passed to the function.**

```
int execl(const char *path, const char *arg1, const char *arg2, ... const char   *argn, (char *) 0);
```

Command line:

```
ls -l
```

```
xueyu@xueyu-VirtualBox:~/下載$ ls -l
總計 10272
-rwxr-xr-x 1 xueyu xueyu     8344  7月 31 14:37 a.out
drwxr-xr-x 2 xueyu xueyu     4096 12月 15  2019 porg01
-rw-rw-r-- 1 xueyu xueyu 10497324 12月 15  2019 porg01-20191215T061538Z-001.zip
-rw-r--r-- 1 xueyu xueyu       86  7月 31 14:37 test.c
```

```
1  main() {
2      printf("Files in Directory are: \n");
3      execl("/bin/ls", "ls", "-l", 0);
4  }
```

```
xueyu@xueyu-VirtualBox:~/下載$ ./a.out
Files in Directory are:
總計 10272
-rwxr-xr-x 1 xueyu xueyu     8344  7月 31 14:37 a.out
drwxr-xr-x 2 xueyu xueyu     4096 12月 15  2019 porg01
-rw-rw-r-- 1 xueyu xueyu 10497324 12月 15  2019 porg01-20191215T061538Z-001.zip
-rw-r--r-- 1 xueyu xueyu       86  7月 31 14:37 test.c
```

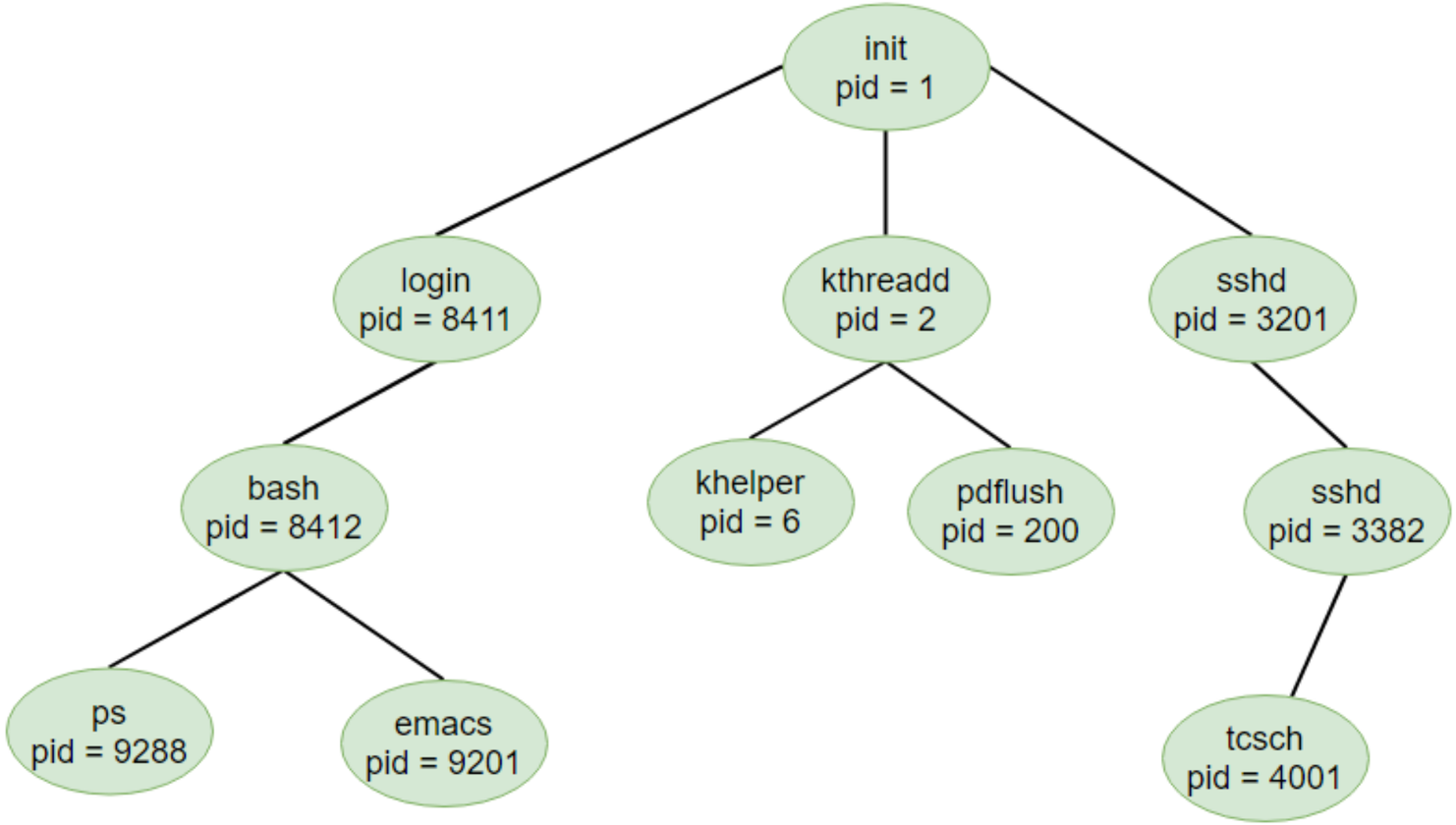
`execl()`,`execlp()`– `exec()` family

- **The routine `execlp()` will perform the same purpose except that it will use environment variable `PATH` to determine which executable to process. Thus a fully qualified path name would not have to be used.**

Process Hierarchies

- **Parent creates a child process, child processes can create its own process**
- **Forms a hierarchy**
  - UNIX calls this a "process group"
- **Windows has no concept of process hierarchy**
  - all processes are created equal

A Tree of Processes in Linux



Creating a Separate Process via Windows API

```
#include <stdio.h>
#include <windows.h>

int main(VOID)
{
STARTUPINFO si;
PROCESS_INFORMATION pi;

    /* allocate memory */
    ZeroMemory(&si, sizeof(si));
    si.cb = sizeof(si);
    ZeroMemory(&pi, sizeof(pi));

    /* create child process */
    if (!CreateProcess(NULL, /* use command line */
     "C:\\WINDOWS\\system32\\mspaint.exe", /* command */
     NULL, /* don't inherit process handle */
     NULL, /* don't inherit thread handle */
     FALSE, /* disable handle inheritance */
     0, /* no creation flags */
     NULL, /* use parent's environment block */
     NULL, /* use parent's existing directory */
     &si,
     &pi))
    {
       fprintf(stderr, "Create Process Failed");
       return -1;
    }
    /* parent will wait for the child to complete */
    WaitForSingleObject(pi.hProcess, INFINITE);
    printf("Child Complete");

    /* close handles */
    CloseHandle(pi.hProcess);
    CloseHandle(pi.hThread);
}
```

Process Termination

- **Process executes last statement or asks the operating system to delete it (`exit()`)**
  - Output data from child to parent (via **`wait()`**)
  - Process' resources are deallocated by operating system
- **Parent may terminate execution of children processes (`abort()`)**
  - Child has exceeded allocated resources
  - Task assigned to child is no longer required
  - If parent is exiting
    - Some operating systems do not allow child to continue if its parent terminates
      - All children terminated - **cascading termination**

Process Termination

- **Wait for termination, returning the pid:**

```
pid_t pid;
int status;
pid = wait(&status);
```

- **If no parent waiting, then terminated process is a zombie**
- **If parent terminated, processes are orphans**

Multiprocess Architecture – Chrome Browser

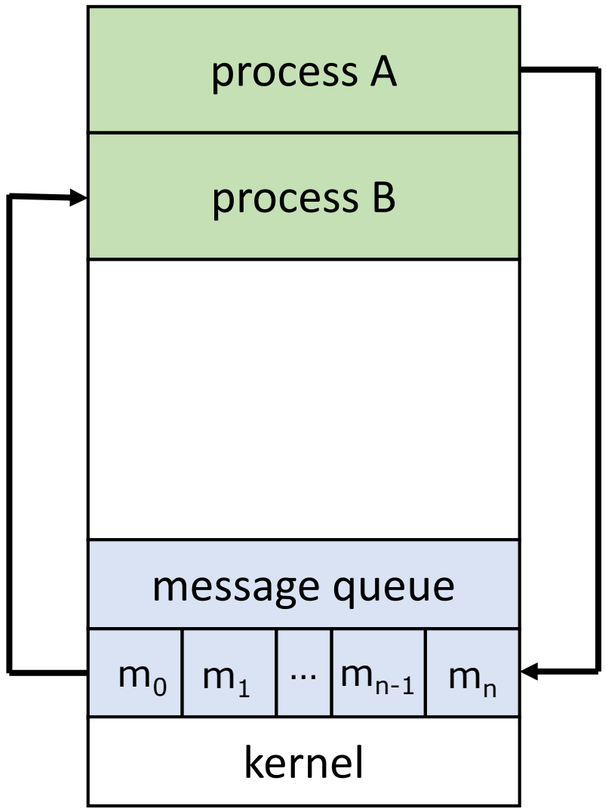
- **Many web browsers ran as single process (some still do)**
  - If one web site causes trouble, entire browser can hang or crash
- **Google Chrome Browser is multiprocess with 3 categories**
  - **Browser** process manages user interface, disk and network I/O
  - **Renderer** process renders web pages, deals with HTML, Javascript, new one for each website opened
    - Runs in **sandbox** restricting disk and network I/O, minimizing effect of security exploits
  - **Plug-in** process for each type of plug-in



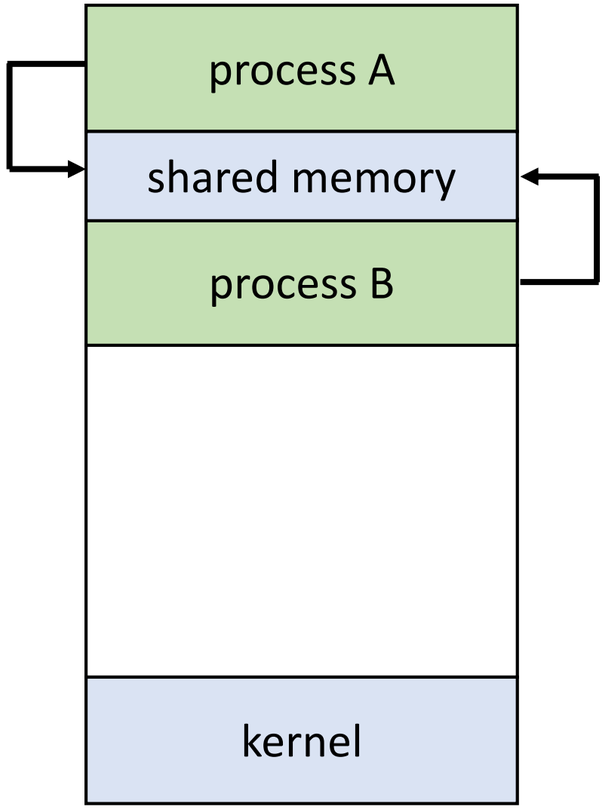
Interprocess Communication

- **Processes within a system may be *independent* or *cooperating***
  - Independent process cannot affect or be affected by the execution of another process
  - Cooperating process can affect or be affected by other processes, including sharing data
- **Reasons for cooperating processes:**
  - Information sharing
  - Computation speedup
  - Modularity
  - Convenience
- **Cooperating processes need interprocess communication (IPC)**
- **Two models of IPC**
  - **Shared memory**
  - **Message passing**

Communications Models



Message passing



Shared memory

Producer-Consumer Problem

- **Paradigm for cooperating processes, *producer* process produces information that is consumed by a *consumer* process**
  - **unbounded-buffer** places no practical limit on the size of the buffer
  - **bounded-buffer** assumes that there is a fixed buffer size

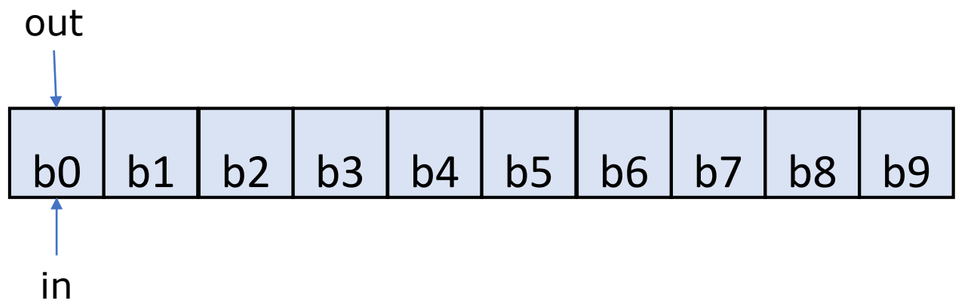
Bounded-Buffer – Shared-Memory Solution

- **Shared data**

```
#define BUFFER_SIZE 10
typedef struct {
    . . .
} item;

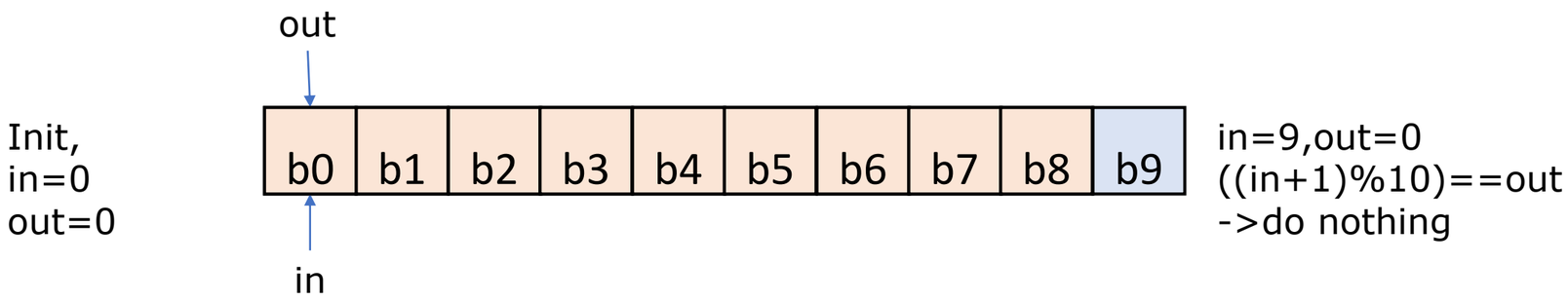
item buffer[BUFFER_SIZE];
int in = 0;
int out = 0;
```

- **Solution is correct, but can only use `BUFFER_SIZE-1` elements**



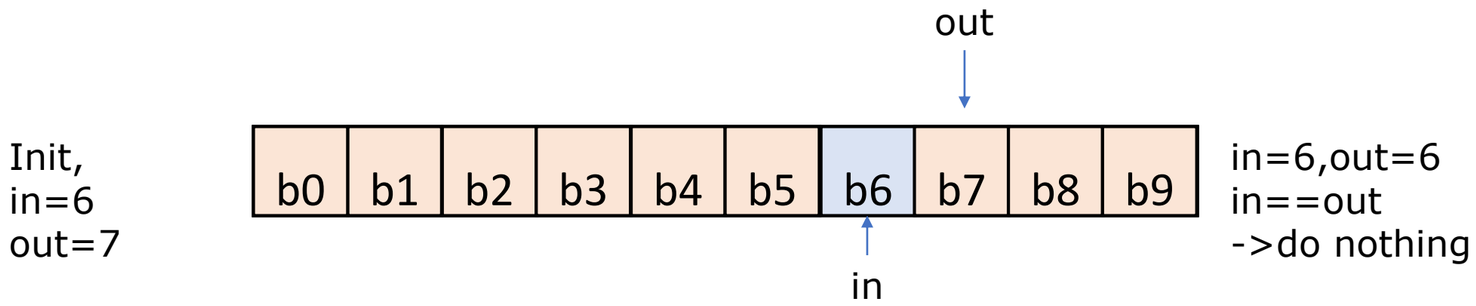
Bounded-Buffer – Producer

```
1  item next_produced;
2  while (true) {
3      /* produce an item in next produced */
4      while (((in + 1) % BUFFER_SIZE) == out)
5          ; /* do nothing */
6      buffer[in] = next_produced;
7      in = (in + 1) % BUFFER_SIZE;
8  }
```



Bounded Buffer – Consumer

```
1  item next_consumed;
2  while (true) {
3      while (in == out)
4          ; /* do nothing */
5      next_consumed = buffer[out];
6      out = (out + 1) % BUFFER_SIZE;
7      /* consume the item in next consumed */
8  }
```



Interprocess Communication – Message Passing

- **Mechanism for processes to communicate and to synchronize their actions**
- **Message system – processes communicate with each other without resorting to shared variables**
- **IPC facility provides two operations:**
  - `send(message)` – message size fixed or variable
  - `receive(message)`
- **If *P* and *Q* wish to communicate, they need to:**
  - establish a ***communication link*** between them
  - exchange messages via send/receive
- **Implementation of communication link**
  - physical (e.g., shared memory, hardware bus)
  - logical (e.g., direct or indirect, synchronous or asynchronous, automatic or explicit buffering)

Implementation Questions

- **How are links established?**
- **Can a link be associated with more than two processes?**
- **How many links can there be between every pair of communicating processes?**
- **What is the capacity of a link?**
- **Is the size of a message that the link can accommodate fixed or variable?**
- **Is a link unidirectional or bi-directional?**

Direct Communication

- **Processes must name each other explicitly:**
  - **`send(P, message)`** – send a message to process **`P`**
  - **`receive(Q, message)`** – receive a message from process **`Q`**

- **Properties of communication link**
  - Links are established automatically
  - A link is associated with exactly one pair of communicating processes
  - Between each pair there exists exactly one link
  - The link may be unidirectional, but is usually bi-directional

Indirect Communication

- **Messages are directed and received from mailboxes (also referred to as ports)**
  - Each mailbox has a unique id
  - Processes can communicate only if they share a mailbox

- **Properties of communication link**
  - Link established only if processes share a common mailbox
  - A link may be associated with many processes
  - Each pair of processes may share several communication links
  - Link may be unidirectional or bi-directional

Indirect Communication

Operations

- create a new mailbox
- send and receive messages through mailbox
- destroy a mailbox

Primitives are defined as:

`send(A, message)` – send a message to mailbox **`A`**

`receive(A, message)` – receive a message from mailbox **`A`**

Indirect Communication

Mailbox sharing

- P_1, P_2, and P_3 share mailbox **`A`**
- P_1, sends; P_2 and P_3 receive
- Who gets the message?

Solutions

- Allow a link to be associated with at most two processes
- Allow only one process at a time to execute a receive operation
- Allow the system to select arbitrarily the receiver. Sender is notified who the receiver was.

Synchronization

- **Message passing may be either blocking or non-blocking**

- **Blocking is considered synchronous**
  - **Blocking send** has the sender block until the message is received
  - **Blocking receive** has the receiver block until a message is available

- **Non-blocking is considered asynchronous**
  - **Non-blocking** send has the sender send the message and continue
  - **Non-blocking** receive has the receiver receive a valid message or null

Synchronization (Cont.)

- **Different combinations possible**
  - If both send and receive are blocking, we have a **rendezvous**
- **Producer-consumer becomes trivial**

```
message next_produced;
while (true) {
    /* produce an item in next_produced */
    send(next_produced);
}
```

```
message next_consumed;
while (true) {
    receive(next_consumed);

    /* consume the item in next_consumed */
}
```

Buffering

Queue of messages attached to the link; implemented in one of three ways

1) Zero capacity – 0 messages

   Sender must wait for receiver (rendezvous)

2) Bounded capacity – finite length of *n* messages

   Sender must wait if link full

3) Unbounded capacity – infinite length

   Sender never waits

Examples of IPC Systems - POSIX

POSIX Shared Memory

- Process first creates shared memory segment

```
shm_fd = shm_open(name, O_CREAT | O_RDRW, 0666);
```

- Also used to open an existing segment to share it
- Set the size of the object

```
ftruncate(shm_fd, 4096);
```

- Now the process could write to the shared memory

```
sprintf(shared_memory, "Writing to shared memory");
```

IPC POSIX Producer

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <fcntl.h>
#include <sys/shm.h>
#include <sys/stat.h>
int main()
{
    /* the size (in bytes) of shared memory object */
    const int SIZE = 4096;
    /* name of the shared memory object */
    const char* name = "OS";
    /* strings written to shared memory */
    const char* message_0 = "Hello";
    const char* message_1 = "World!";
    /* shared memory file descriptor */
    int shm_fd;
    /* pointer to shared memory obect */
    void* ptr;
    /* create the shared memory object */
    shm_fd = shm_open(name, O_CREAT | O_RDWR, 0666);
    /* configure the size of the shared memory object */
    ftruncate(shm_fd, SIZE);
    /* memory map the shared memory object */
    ptr = mmap(0, SIZE, PROT_WRITE, MAP_SHARED, shm_fd, 0);
    /* write to the shared memory object */
    sprintf(ptr, "%s", message_0);
    ptr += strlen(message_0);
    sprintf(ptr, "%s", message_1);
    ptr += strlen(message_1);
    return 0;
}
```

IPC POSIX Consumer

```c
#include <stdio.h>
#include <stdlib.h>
#include <fcntl.h>
#include <sys/shm.h>
#include <sys/stat.h>

int main()
{
    /* the size (in bytes) of shared memory object */
    const int SIZE = 4096;
    /* name of the shared memory object */
    const char* name = "OS";
    /* shared memory file descriptor */
    int shm_fd;
    /* pointer to shared memory object */
    void* ptr;
    /* open the shared memory object */
    shm_fd = shm_open(name, O_RDONLY, 0666);
    /* memory map the shared memory object */
    ptr = mmap(0, SIZE, PROT_READ, MAP_SHARED, shm_fd, 0);
    /* read from the shared memory object */
    printf("%s", (char*)ptr);
    /* remove the shared memory object */
    shm_unlink(name);
    return 0;
}
```

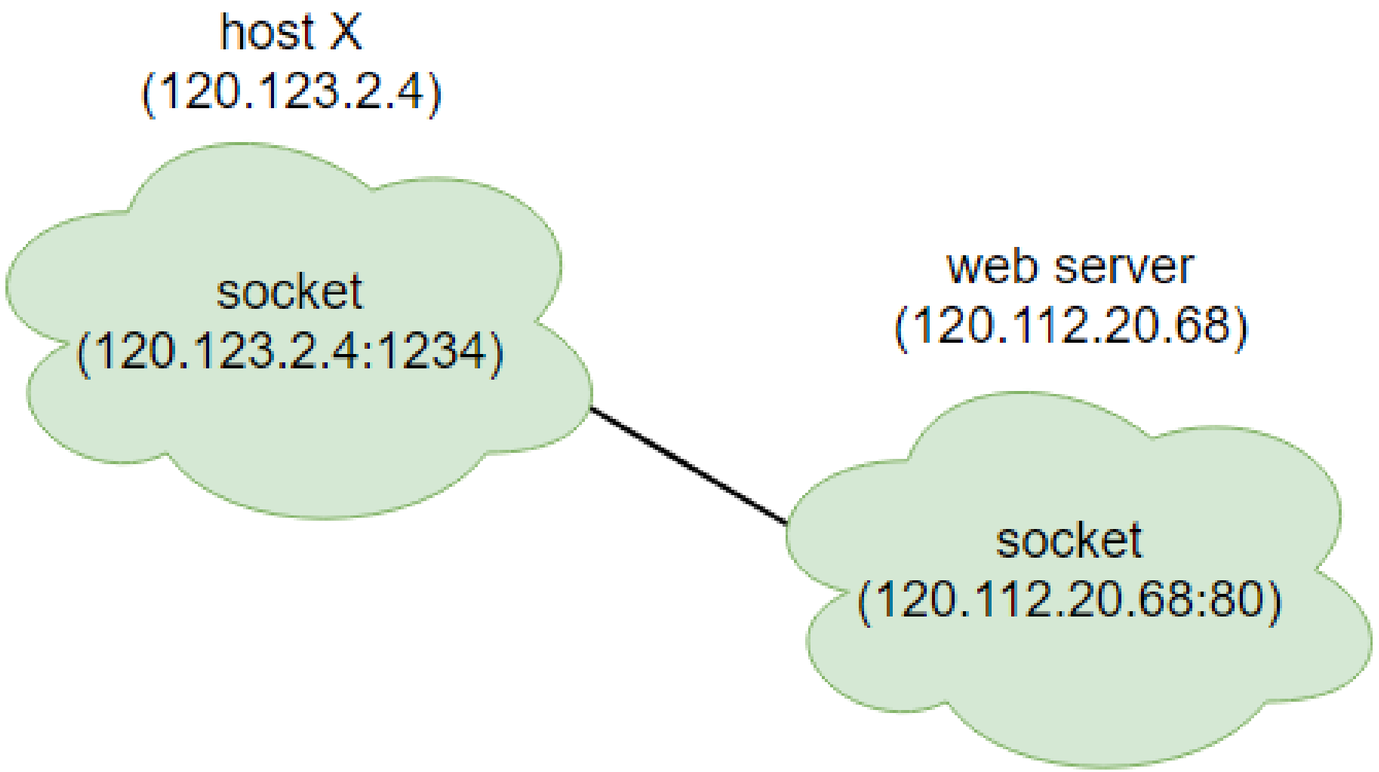
Communications in Client-Server Systems

- **Socket**
- **Remote Procedure Call**
- **Remote Method Invocation (Java)**
- **Pipe**

Socket

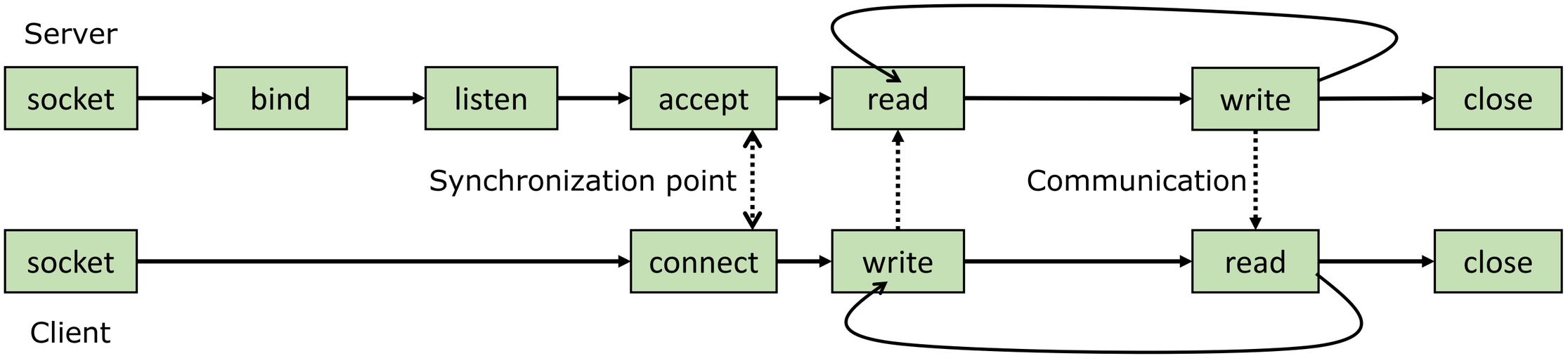
- A socket is defined as an endpoint for communication
- Concatenation of IP address and port – a number included at start of message packet to differentiate network services on a host
- The socket 161.25.19.8:1625 refers to port 1625 on host 161.25.19.8
- Communication consists between a pair of sockets
- All ports below 1024 are *well known*, used for standard services
- Special IP address 127.0.0.1 (loopback) to refer to system on which process is running

Socket Communication

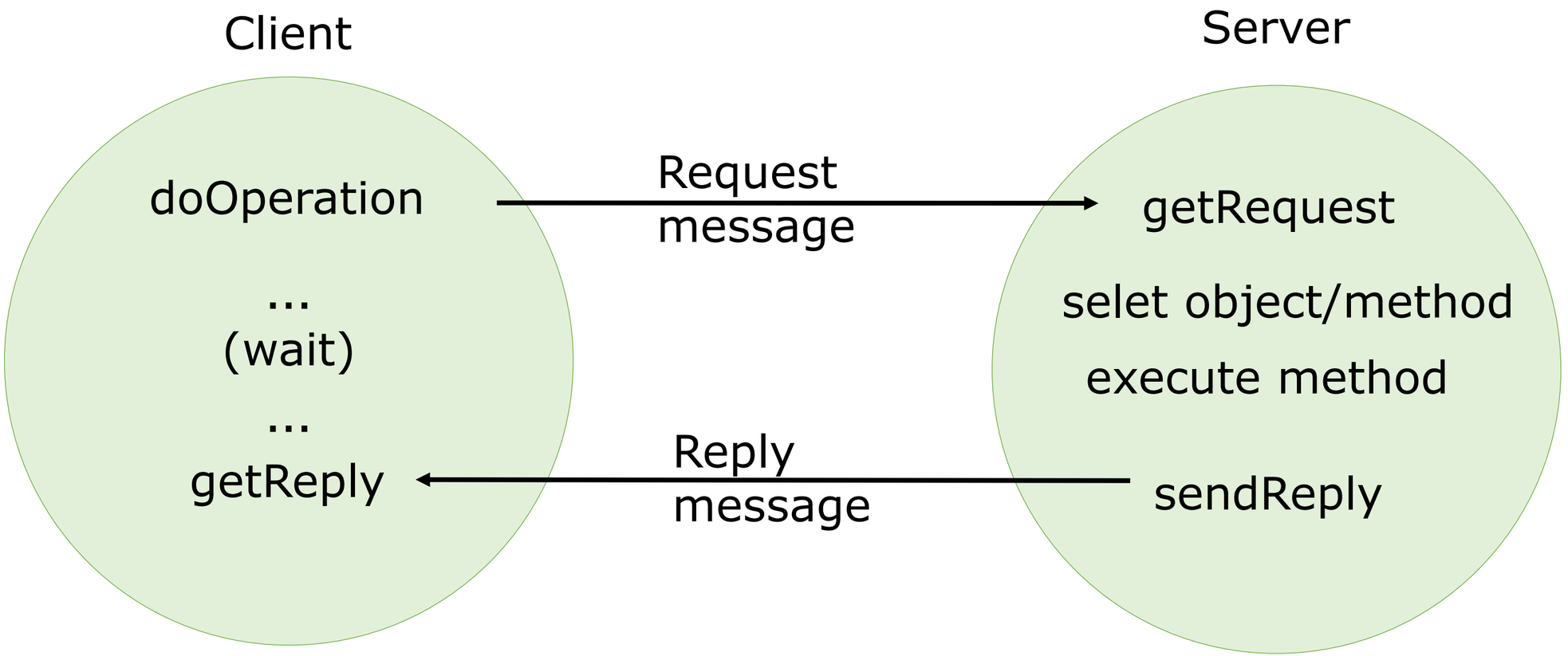


Message-Passing Interface

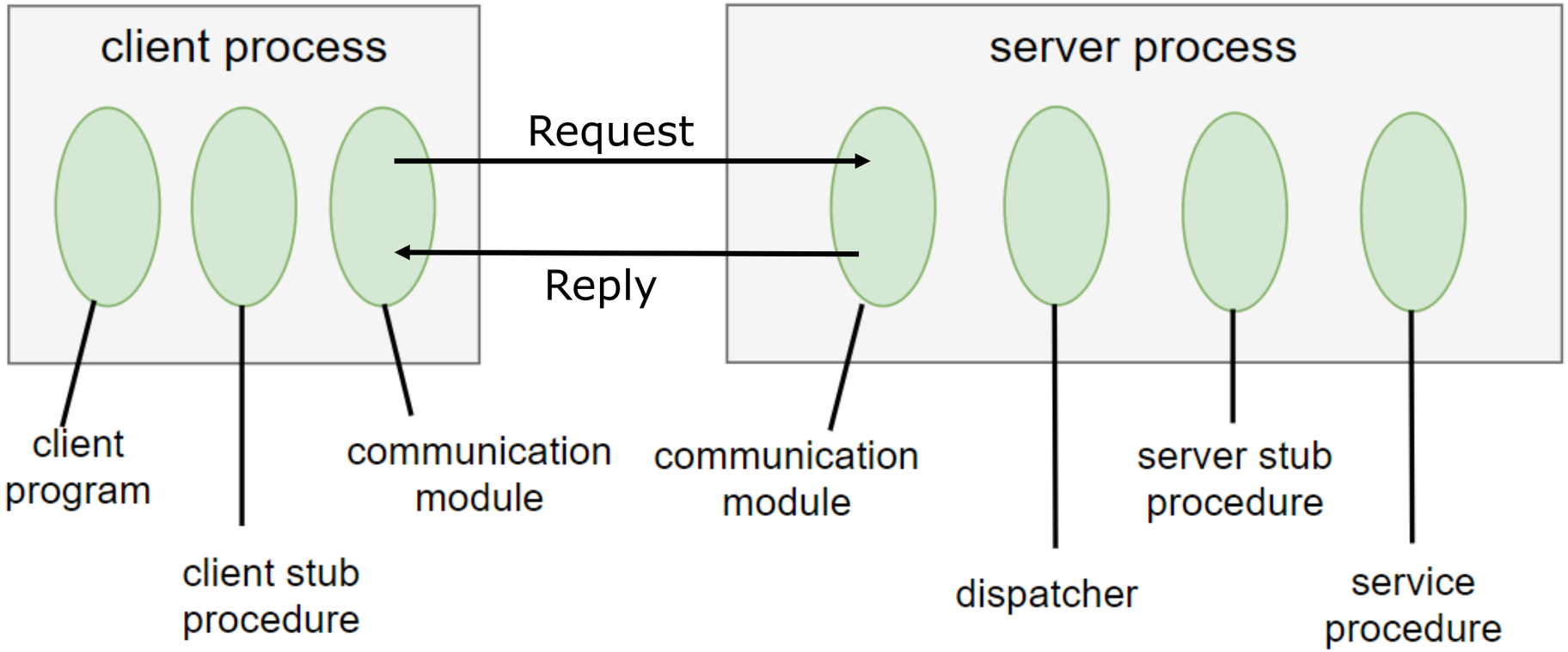
Figure 4-15. Connection-oriented communication pattern using sockets.



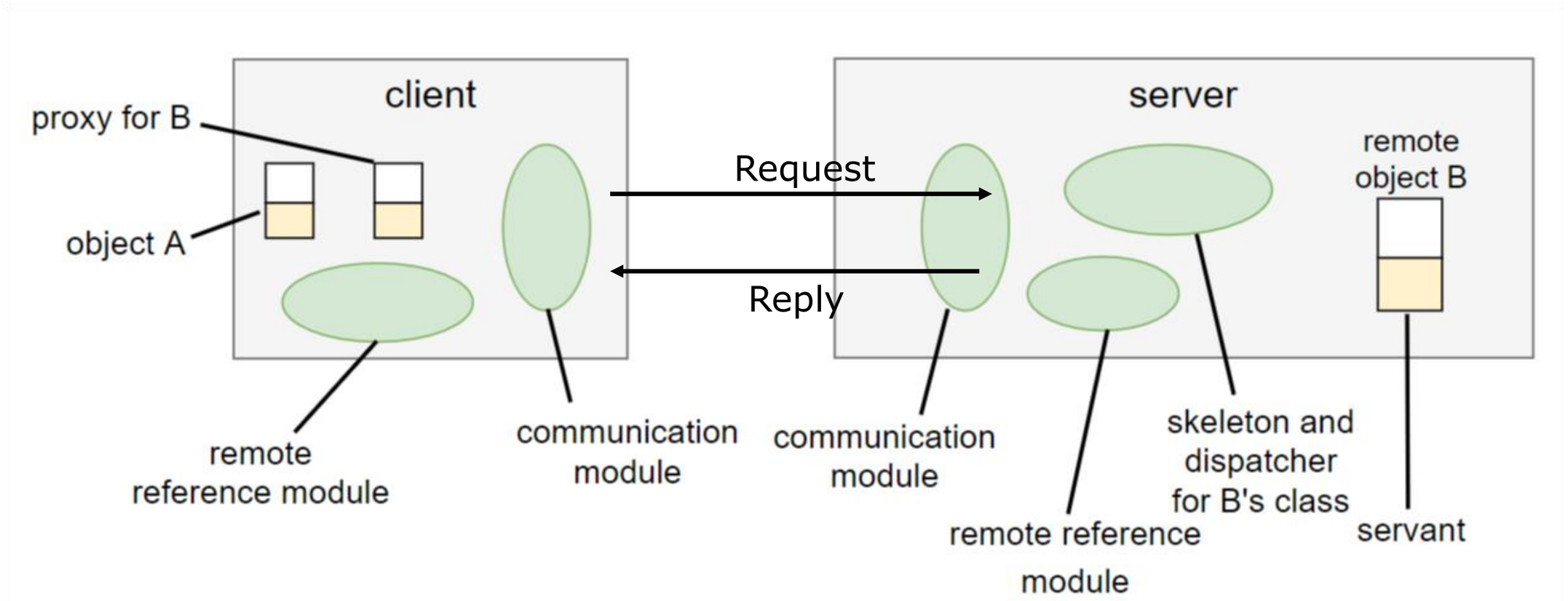
Request-reply communication



Remote Procedure Call



Remote Method Invocation



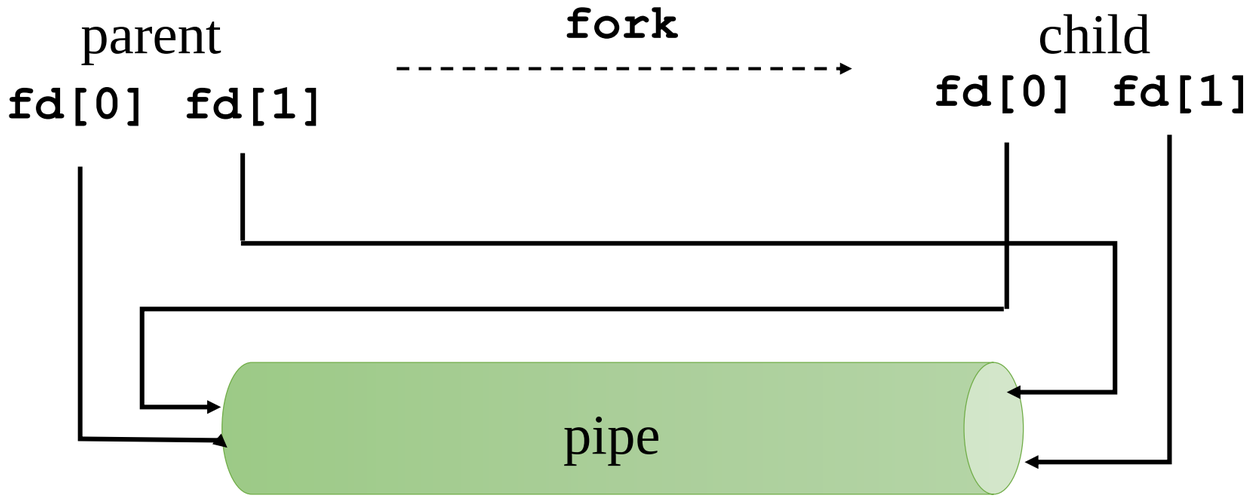
Pipe

- **Acts as a conduit allowing two processes to communicate**

- **Issues**
  - Is communication unidirectional or bidirectional?
  - Must there exist a relationship (i.e. ***parent-child***) between the communicating processes?

Ordinary Pipe

- Ordinary pipes allow communication in standard producer-consumer style
- Producer writes to one end (the write-end of the pipe)
- Consumer reads from the other end (the read-end of the pipe)
- Ordinary pipes are therefore unidirectional
- Require parent-child relationship between communicating processes
- Windows calls these anonymous pipes



Named Pipe

- **Named pipes are more powerful than ordinary pipes**
- **Communication is bidirectional**
- **No parent-child relationship is necessary between the communicating processes**
- **Several processes can use the named pipe for communication**
- **Provided on both UNIX and Windows systems**



Thanks for listening